Lab Exercise 17- Creating Secrets in Kubernetes

Viraj bhidola
500121825
B2 DevOps

Creating secrets in Kubernetes is an essential practice for managing sensitive information such as passwords, OAuth tokens, and SSH keys. Let's go through a lab exercise that demonstrates how to create different types of secrets in Kubernetes.

Step 1: Create a Generic Secret

Create a secret from literal values:

```
kubectl create secret generic my-secret --from-literal=username=admin --from-literal=password='P@ssword'
```

```
PS C:\Users\ASUS> kubectl create secret generic my-secret --from-literal=use
rname=admin --from-literal=password='P@ssw0rd'
secret/my-secret created
PS C:\Users\ASUS>
```

Verify the secret:

```
kubectl get secrets my-secret -o yaml
```

```
PS C:\Users\ASUS> kubectl get secrets my-secret -o yaml
apiVersion: v1
data:
  password: UEBzc3cwcmQ=
  username: YWRtaW4=
kind: Secret
metadata:
  creationTimestamp: "2026-03-05T08:54:18Z"
  name: my-secret
  namespace: default
  resourceVersion: "21046"
  uid: c348bc81-1df0-4c41-baf0-7fd9302585e7
type: Opaque
PS C:\Users\ASUS>
```

The output will be base64 encoded. To decode, you can use:

```
echo -n “P@ssword” | base64
```

```
PS C:\Users\ASUS> [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("
P@ssw0rd"))
UEBzc3cwcmQ=
```

Step 2: Create a Secret from a File

Create a file with your sensitive data:

```
echo -n 'my-username' > ./username.txt



echo -n 'my-password' > ./password.txt
```

```
PS C:\Users\ASUS> echo my-username > username.txt
PS C:\Users\ASUS> echo my-password > password.txt
```

Create a secret from the file:

```
kubectl create secret generic my-file-secret --from-file=username=./username.txt --from-
file=password=./password.txt
```

```
PS C:\Users\ASUS> kubectl create secret generic my-file-secret --from-file=u
sername=./username.txt --from-file=password=./password.txt
secret/my-file-secret created
PS C:\Users\ASUS>
```

Verify the secret:

```
kubectl get secrets my-file-secret -o yaml
```

```
PS C:\Users\ASUS> kubectl get secrets my-file-secret -o yaml
apiVersion: v1
data:
  password: //5tAHkALQBwAGEAcwBzAHcAbwByAGQADQAKAA==
  username: //5tAHkALQB1AHMAZQByAG4AYQBtAGUADQAKAA==
kind: Secret
metadata:
  creationTimestamp: "2026-03-05T08:57:35Z"
  name: my-file-secret
  namespace: default
  resourceVersion: "21209"
  uid: 4e98a231-78f7-40a8-b523-1f3ca2c555ea
type: Opaque
PS C:\Users\ASUS>
```

Step 3: Create a Docker Registry Secret

Create a Docker registry secret:

```
kubectl create secret docker-registry my-registry-secret --docker-username=<your-username> --docker-password=<your-password> --docker-email=<your-email> --docker-server=<your-docker-server>
```

```
PS C:\Users\ASUS> kubectl create secret docker-registry my-registry-secret --docker-username=testuser --docker-password=testpass --docker-email=test@example.com --docker-server=docker.io
secret/my-registry-secret created
PS C:\Users\ASUS>
```

Verify the secret:

```
kubectl get secrets my-registry-secret -o yaml
```

```
PS C:\Users\ASUS> kubectl get secrets my-registry-secret -o yaml
apiVersion: v1
data:
  .dockerconfigjson: eyJhdXRocyI6eyJkb2NrZXIuaW8iOnsidXNlcm5hbWUiOiJ0ZXN0dXN
lciIsInBhc3N3b3JkIjoidGVzdHBhc3MiLCJlbWFpbCI6InRlc3RAZXhhbXBsZS5jb20iLCJhdXR
oIjoiZEdWemRIVnpaWEk2ZEdWemRIQmhjM009In19fQ==
kind: Secret
metadata:
  creationTimestamp: "2026-03-05T09:00:07Z"
  name: my-registry-secret
  namespace: default
  resourceVersion: "21336"
  uid: faf7d30a-c449-4330-9eae-708969ef2326
type: kubernetes.io/dockerconfigjson
PS C:\Users\ASUS>
```

Step 4: Create a Secret Using a YAML Manifest

Create a YAML file for the secret:

Create a file named **my-secret.yaml** with the following content:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-manifest-secret
type: Opaque
data:
  username: YWRtaW4=    # base64 encoded value of 'admin'
  password: UEBzc3cwcmQ=  # base64 encoded value of 'P@ssword'
```

```
File    Edit    View

apiVersion: v1
kind: Secret
metadata:
  name: my-manifest-secret
type: Opaque
data:
  username: YWRtaW4=
  password: UEBzc3cwcmQ=
```

Apply the YAML file:

```
kubectl apply -f my-manifest-secret.yaml
```

```
PS C:\Users\ASUS> notepad my-manifest-secret.yaml
PS C:\Users\ASUS> kubectl apply -f my-manifest-secret.yaml
secret/my-manifest-secret created
PS C:\Users\ASUS>
```

Verify the secret:

```
kubectl get secrets my-manifest-secret -o yaml
```

```
PS C:\Users\ASUS> kubectl get secrets my-manifest-secret -o yaml
apiVersion: v1
data:
  password: UEBzc3cwcmQ=
  username: YWRtaW4=
kind: Secret
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"password":"UEBzc3cwcmQ=","username":"YWRta
W4="},"kind":"Secret","metadata":{"annotations":{},"name":"my-manifest-secre
t","namespace":"default"},"type":"Opaque"}
  creationTimestamp: "2026-03-05T09:01:00Z"
  name: my-manifest-secret
  namespace: default
  resourceVersion: "21380"
  uid: afebe859-bbf7-463e-8027-f04947474cca
type: Opaque
PS C:\Users\ASUS>
```

Step 5: Use Secrets in Pods

Create a pod that uses the secret:

Create a file named **pod-with-secret.yaml** with the following content:

```
apiVersion: v1
```

```
kind: Pod
metadata:
  name: pod-with-secret
spec:
  containers:
  - name: mycontainer
    image: nginx
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: username
    - name: PASSWORD
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: password
```

File Edit View

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-secret
spec:
  containers:
  - name: mycontainer
    image: nginx
    env:
    - name: USERNAME
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: username
    - name: PASSWORD
      valueFrom:
        secretKeyRef:
          name: my-secret
          key: password
```

Apply the YAML file:

```
kubectl apply -f pod-with-secret.yaml
```

```
PS C:\Users\ASUS> notepad pod-with-secret.yaml
PS C:\Users\ASUS> kubectl apply -f pod-with-secret.yaml
pod/pod-with-secret created
PS C:\Users\ASUS>
```

Verify the pod is running and the secret is being used:

```
kubectl exec pod-with-secret -- printenv | grep -E 'USERNAME|PASSWORD'
```

Step 7: Clean Up

Delete the secrets:

```
kubectl delete secret my-secret
kubectl delete secret my-file-secret
kubectl delete secret my-registry-secret
kubectl delete secret my-manifest-secret
```

Delete the pod:

```
kubectl delete pod pod-with-secret
```

```
PS C:\Users\ASUS> kubectl delete secret my-secret
secret "my-secret" deleted from default namespace
PS C:\Users\ASUS> kubectl delete secret my-file-secret
secret "my-file-secret" deleted from default namespace
PS C:\Users\ASUS> kubectl delete secret my-registry-secret
secret "my-registry-secret" deleted from default namespace
PS C:\Users\ASUS> kubectl delete secret my-manifest-secret
secret "my-manifest-secret" deleted from default namespace
PS C:\Users\ASUS> kubectl delete pod pod-with-secret
pod "pod-with-secret" deleted from default namespace
PS C:\Users\ASUS>
```